

第2章 运算符与表达式

建议学时:3-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

本章学习目标

- 掌握 Java 算术运算与 C 的三点根本差异:有符号溢出是"回绕"而非 UB、整数除零抛异常、浮点除零得 Infinity/NaN
- 掌握 / 与 % 的取整方向,会用 Math.floorDiv/floorMod 获得向下取整语义
- 掌握复合赋值运算符的"隐藏强转",避免 `s += 1` 与 `s = s + 1` 的迷惑
- 掌握全套位运算与三种移位(`<<` `>>` `>>>`),会用标志位技术
- 掌握 `&&/||` 的短路求值与"操作数必须是 boolean"的强约束
- 掌握 `==` 的值/引用语义、浮点比较的正确姿势、instanceof 模式匹配
- 掌握完整优先级表与"操作数从左到右求值"的规定(C 的求值顺序是 UB!)

从 C 到 Java:本章主题如何迁移

运算符是 C 程序员最有"熟悉感"也最容易翻车的一章:符号几乎全认识,语义却在几处关键点上完全不同。

第一,溢出的处理哲学不同。C 里 `int` 溢出是未定义行为,编译器可以"做任何事";Java 把行为固化下来:有符号整数溢出是 **静默回绕** (wrap-around),整数除零是抛异常,浮点除零是 Infinity。看似少了"自由",实则是生产安全的巨大胜利——行为可预测、可测试。

第二,真假值的强约束。C 的 `if (x)` 接受任意整数;Java 的 `&&/||/!` 只接受 boolean。`if (n = 5)` 这种"赋值当比较"的经典 C 事故,在 Java 里直接编译失败。代价是写法上必须显式比较;收益是一类 bug 从根上消失。

第三,求值顺序被规定死了。C 里 `f() + g()` 先调谁未定义;Java 规定操作数一律从左到右求值。本章的优先级表与 C 几乎相同,但"同一表达式不同编译器结果不同"这种事不再发生。

本章策略:先过一遍与 C 相同的部分(快速),然后把精力集中在上述差异与 `>>>`、instanceof、Math 工具这些 C 没有的东西上。

前置知识自查

- C 的算术运算符、自增自减、位运算(`&` `|` `^` `~` `<<` `>>`)、逻辑运算与短路
- C 的运算符优先级与结合性(口诀:单目 > 算术 > 移位 > 关系 > 位 > 逻辑 > 三目 > 赋值)
- C 的整数除法截断与取模符号规则;浮点精度问题
- C 的 printf 占位符与条件表达式 ?:

本章学习路线

1. **第一阶段(约 1.5 小时,扫差异):**重点阅读 §2.1 溢出/除零/取模与 §2.4 短路,与 C 逐条对照;其余快速过
2. **第二阶段(约 1.5 小时,动手写):**运行 §2.2-§2.6 的全部示例,亲手验证每个输出,把"猜答案→对答案"做成小测验
3. **第三阶段(实验,约 1 小时):**独立完成章末实验"表达式计算器",用自测题验收
4. **验收标准:**不看资料能预测:溢出结果、取模符号、除零行为、短路是否执行右侧、复合赋值是否截断

本章知识导图

- §2.1 算术运算:溢出回绕 / 除零 / 除法与取模 / Math.floorDiv
- §2.2 自增自减与复合赋值(隐藏强转陷阱)
- §2.3 位运算与移位(>>> 逻辑右移 / 移位距离取模 / 标志位场景)
- §2.4 逻辑运算与短路(boolean 强约束 / 判空模式)
- §2.5 比较与相等(== 值/引用 / 浮点比较姿势 / instanceof 模式匹配)
- §2.6 三元、优先级与求值顺序(完整优先级表 / 从左到右求值)
- §2.7 字符串拼接 + 与 Math 工具速览

§2.1 算术运算:溢出、除零与取模

2.1.1 溢出:C 的 UB 与 Java 的回绕

C 标准规定有符号整数溢出是未定义行为,编译器可能优化出任何结果;Java 规定:有符号整数运算按补码回绕,即 `Integer.MAX_VALUE + 1` 恒等于 `Integer.MIN_VALUE`。行为确定、可测试,但不等于“不会错”——回绕值常常静默地通过后续检查,生产上应在运算前做范围检查或用 `Math.addExact` 等精确运算方法(溢出时抛异常)。

示例 2-1 溢出与除零

```
public class ArithDemo {
    public static void main(String[] args) {
        // 1) 整型溢出:C 是 UB,Java 是“静默回绕”
        int max = Integer.MAX_VALUE;
        System.out.println(max + 1);           // -2147483648:回绕
        System.out.println(max + 1L);          // 2147483648L:先提升为 long,安全
        System.out.println(Math.addExact(max, 1)); // 抛 ArithmeticException

        // 2) 整数除以零:抛异常(C 是 UB/直接崩溃)
        // int x = 1 / 0; // ArithmeticException: / by zero

        // 3) 浮点除以零:不抛异常,得无穷或 NaN
        System.out.println(1.0 / 0);           // Infinity
        System.out.println(-1.0 / 0);          // -Infinity
        System.out.println(0.0 / 0);           // NaN

        // 4) NaN 不等于任何数,包括自己
        double nan = 0.0 / 0.0;
        System.out.println(nan == nan);        // false!
        System.out.println(Double.isNaN(nan)); // true:唯一的正确判断方式
    }
}
```

⚠ 易错点 1:回绕值“看起来合理”

`Integer.MAX_VALUE + 1` 得到 `-2147483648`,如果代码接下来用 `if (sum > 0)` 判断正负,这个负数会静默通过边界检查,产生“查不出来的 bug”。生产防护:金额与计数器用 `long`;必须 `int` 且可能溢出时用 `Math.addExact/subtractExact/multiplyExact` (溢出即抛异常,至少能在测试期暴露)。

2.1.2 除法与取模:与 C 完全一致,但多了工具

Java 的 `/` 与 `%` 对整数的语义与 C 完全一致:向零取整,取模符号跟随被除数。C 没有的工具是 `Math.floorDiv / Math.floorMod`:提供“向下取整”语义(Python 风格),循环索引、环形缓冲等场景常用。

示例 2-2 除法与取模

```
public class DivModDemo {
    public static void main(String[] args) {
        // Java 的 / 和 %:向零取整(与 C 完全一致)
        System.out.println(7 / 2);           // 3
        System.out.println(-7 / 2);          // -3(向零,不是向下)
        System.out.println(7 % 2);           // 1
        System.out.println(-7 % 2);          // -1(符号跟随被除数,与 C 一致)
        System.out.println(7 % -2);          // 1

        // Math.floorMod/floorDiv:向下取整语义(相当于 Python 的 // 与 %)
        System.out.println(Math.floorDiv(-7, 2)); // -4(向下)
        System.out.println(Math.floorMod(-7, 2)); // 1(恒为非负)

        // 浮点取模也可用 %
        System.out.println(7.5 % 2);          // 1.5
    }
}
```

语义	C 的写法	Java 的写法
有符号整数溢出	未定义行为(编译器任意发挥)	补码回绕,行为确定;可用 Math.addExact 检测
整数除零	UB(通常崩溃)	抛 ArithmeticException,可捕获处理
浮点除零	实现定义(通常 ±Inf)	明确为 Infinity / NaN
整除与取模	向零取整,符号跟随被除数	相同;另有 floorDiv/floorMod 提供向下取整
求幂	pow(x, y)(double)	Math.pow(double, double);整型幂需自写或 BigInteger

 **工程建议:取模的正确用法**

环形队列、哈希桶、时间换算等场景中,需要"结果恒为非负"时用 `Math.floorMod`,而不是 `%`。例如按 24 小时制换算昨天:今天是 3 点, `(3 - 5) % 24` 得 -2,而 `Math.floorMod(3 - 5, 24)` 得 22——后者才是"昨天 22 点"。

§2.2 自增自减与复合赋值

2.2.1 前置与后置:与 C 相同,但要小心表达式

`++i / --i` (先变后用)与 `i++ / i--` (先用后变)的语义与 C 完全一致,而且同一表达式内的求值顺序是确定的(从左到右),不会出现 C 的 `i = i++ + ++i` 那种 UB。Java 风格建议:自增自减主要用于独立语句与循环,避免塞进复杂表达式——这是团队规范与 C 老手的共同经验。

2.2.2 复合赋值的隐藏强转(经典陷阱)

示例 2-3 复合赋值陷阱

```

public class CompoundDemo {
    public static void main(String[] args) {
        short s = 1;
        s += 1;           // 合法! s += 1 等价于 s = (short)(s + 1), 自带强转
        // s = s + 1;      // 编译错误: int 不能赋给 short

        int i = 5;
        i += 0.9;          // 合法: 复合赋值做隐式收窄, i 变成 5 (小数被截断)
        System.out.println(s + " " + i);    // 2 5

        // 自增自减: 与 C 相同, 先变后用 / 先用后变
        int a = 5;
        int b = a++;        // b = 5, a = 6
        int c = ++a;        // c = 7, a = 7
        System.out.println(a + " " + b + " " + c);    // 7 5 7

        // 浮点累加陷阱: 不要用 == 判断浮点结束
        double x = 0;
        // for (x = 0; x != 1; x += 0.1) {} // 0.1 不精确, 可能死循环
        for (x = 0; x < 1; x += 0.1) {}           // 用 < 而不是 !=
        System.out.println(x);                     // 1.0999999999999999
    }
}

```

⚠ 易错点 2: `i += 0.9` 不报错, 结果却是 5

复合赋值 `E1 op= E2` 等价于 `E1 = (T)(E1 op E2)`, 其中 T 是 E1 的类型——**自带一次强转**。所以 `int i = 5; i += 0.9;` 编译通过且结果为 5 (截断), 而 `i = i + 0.9` 编译错误。生产上遇到“浮点混进整数累加”要特别警惕: 类型检查放过了它, 精度却丢了。

§2.3 位运算与移位

2.3.1 全套位运算: C 有的都有, C 没有的也有

Java 支持 C 的全部位运算符: `&` (与)、`|` (或)、`^` (异或)、`~` (取反), 以及 `<<` (左移)、`>>` (算术右移, 补符号位)。C 没有的是 `>>>` (**无符号右移, 高位补 0**)——因为 C 没有 unsigned 与 signed 之争以外的第三种选择, Java 的 `int` 全是有符号, `>>>` 让“只关心低位”的场景不必再折腾符号位。

示例 2-4 位运算与三种移位

```

public class BitDemo {
    public static void main(String[] args) {
        int a = 0b1100;    // 12
        int b = 0b1010;    // 10
        System.out.println(a & b);    // 8 (1000) 按位与
        System.out.println(a | b);    // 14 (1110) 按位或
        System.out.println(a ^ b);    // 6 (0110) 按位异或
        System.out.println(~a);       // -13 (按位取反, 含符号位)

        // 移位: << 左移 (等效乘 2^n); >> 算术右移 (补符号位); >>> 逻辑右移 (补 0)
        int n = 1;
        System.out.println(n << 4);    // 16: 左移 4 位 = 乘 16
        System.out.println(-16 >> 2);  // -4: 符号位保留
        System.out.println(-16 >>> 2); // 1073741820: 高位补 0

        // 移位距离按 32(int)/64(long) 取模: 1 << 32 等于 1 << 0
        System.out.println(1 << 32);   // 1 (不是 0!)
        System.out.println(1 << 33);   // 2 (等价 1 << 1)
    }
}

```

⚠ 易错点 3:移位距离取模

Java 规定 `int` 的移位距离按 32 取模、`long` 按 64 取模：`1 << 32` 实际是 `1 << 0`，结果是 1。C 中 `1 << 32` 是 UB；Java 里它是“确定但违反直觉”的值。循环中用 `i < 32` 做掩码生成时，`1 << i` 在 `i=32` 时悄悄回到 1——建议循环上限写成 `i < 32` 且掩码用 `1L << i` (64 位)或用 `Math.floorMod` 显式控制。

2.3.2 生产场景:标志位

示例 2-5 权限标志位(C 老手艺的 Java 版)

```
public class FlagDemo {
    // 用一个 int 的多个位保存多个开关:只读/可写/可执行
    static final int READ = 1 << 0;    // 0001
    static final int WRITE = 1 << 1;    // 0010
    static final int EXEC = 1 << 2;     // 0100

    public static void main(String[] args) {
        int perm = READ | WRITE;        // 置位:读 + 写 = 0011
        System.out.println((perm & READ) != 0); // true:检查某位
        System.out.println((perm & EXEC) != 0); // false
        perm |= EXEC;                   // 追加执行权限:0111
        perm &= ~WRITE;                 // 清除写权限:0101
        System.out.println(Integer.toBinaryString(perm)); // 101

        // 交换两个数(教学演示,生产中直接用临时变量更清晰)
        int x = 3, y = 5;
        x ^= y; y ^= x; x ^= y;
        System.out.println(x + " " + y); // 5 3
    }
}
```

🚀 工程建议:位运算优先用于“底层与性能敏感”代码

标志位、哈希、掩码、颜色通道(ARGB)是位运算的用武之地;但业务代码中能用枚举与布尔字段表达就优先用枚举——第1章讲过,可读性与类型安全更重要。位运算出现在:协议解析、位图索引、权限位、整数哈希。

§2.4 逻辑运算与短路

2.4.1 操作数必须是 boolean

`&&` (短路与)、`||` (短路或)、`!` (非)与 C 的语义相同,但要求两侧是 `boolean`。`if (x)` (`x` 为整数)在 Java 中编译失败,必须写成 `if (x != 0)`。这不是麻烦,而是把“非 0 即真”的隐式转换从语言里拿掉了——C 程序里 `if (ptr = malloc(...))` 这类 bug 在 Java 里不可能发生。

示例 2-6 短路求值与判空模式

```

public class ShortCircuitDemo {
    public static void main(String[] args) {
        // 短路:左侧已决定结果时,右侧不执行
        int n = 0;
        boolean ok = (n != 0) && (10 / n > 1); // 左侧 false,不会除零
        System.out.println(ok);                // false

        // 经典判空模式:C 里是 if (s != NULL && strlen(s) > 3)
        String s = null;
        if (s != null && s.length() > 3) {
            System.out.println("长度大于 3");
        }

        // || 同样短路:右侧函数不会执行
        boolean r = true || expensive();
        System.out.println(r);

        // 非短路版本 & | 也支持(不常用):两侧都会执行
        // boolean bad = false & expensive(); // 两侧都算
    }

    static boolean expensive() {
        System.out.println("不会执行到这里");
        return false;
    }
}

```

⚠ 易错点 4:把 & 当 && 用

& 对 boolean 也合法,但不短路: `s != null & s.length() > 3` 在 `s` 为 `null` 时照样调用 `s.length()`,直接 `NullPointerException`。判空 + 后续操作必须用 `&&`;同理 `|` 与 `||`。生产上 `&/|` 只用于位运算(整数),不要用来做逻辑判断。

§2.5 比较与相等:== 的两种世界

2.5.1 值比较与引用比较

Java 的 `==` 语义取决于操作数类型:基本类型比较值,引用类型比较"是否指向同一对象"。C 程序员最容易踩的坑:用 `==` 比较 `String` 或包装类(第1章已详述),这里补充浮点比较的正确姿势——`==` 对浮点"精确到每一位",而 `0.1 + 0.2` 这种运算结果与字面量 `0.3` 的二进制表示不同,直接 `==` 是 `false`。

示例 2-7 浮点比较的正确姿势

```

public class FloatCompareDemo {
    public static void main(String[] args) {
        // 0.1 + 0.2 != 0.3: 二进制浮点不精确(与 C 相同, 但 Java 更常踩中)
        double a = 0.1 + 0.2;
        System.out.println(a);           // 0.30000000000000004
        System.out.println(a == 0.3);    // false!

        // 生产姿势一: 误差范围(epsilon)比较
        double eps = 1e-9;
        System.out.println(Math.abs(a - 0.3) < eps); // true

        // 生产姿势二: 金额用整数(以分为单位)比较, 天然精确
        long centsA = 10, centsB = 20, centsSum = 30;
        System.out.println(centsA + centsB == centsSum); // true

        // 特殊值: NaN 不等于任何数; 正负零相等
        System.out.println(Double.NaN == Double.NaN);    // false
        System.out.println(0.0 == -0.0);                // true
        System.out.println(Double.POSITIVE_INFINITY > 1e300); // true
    }
}

```

2.5.2 instanceof: 类型判断与模式匹配

C 用 `switch` 配合枚举或函数指针表模拟"运行时类型分派"; Java 有 `instanceof` 运算符。Java 16 起支持**模式匹配**: `o instanceof String s` 在判断为真的同时声明变量 `s`, 省去一次强转——这是生产代码中最常用的新写法之一。

示例 2-8 instanceof 模式匹配

```

public class InstanceofDemo {
    public static void main(String[] args) {
        Object o = "hello";
        System.out.println(o instanceof String); // true

        // 旧写法: 判断 + 强转(C 风格)
        if (o instanceof String) {
            String s = (String) o;           // 强转
            System.out.println(s.toUpperCase());
        }

        // 新模式匹配(Java 16+): 判断即声明, 无需强转
        if (o instanceof String s && s.length() > 3) {
            System.out.println(s.toUpperCase()); // HELLO
        }
        // 注意: null instanceof 任何类型都是 false, 天然防空
        System.out.println(null instanceof String); // false
    }
}

```

§2.6 三元、优先级与求值顺序

2.6.1 三元运算符 ?:

语法与 C 相同: 条件 ? 表达式A : 表达式B, **只计算选中的分支**。Java 对两个分支的类型有统一要求(自动找公共类型), `int ? int : double` 的 `int` 会被提升为 `double`, 不要惊讶于类型变化。

2.6.2 完整优先级表(从高到低)

优先级	运算符	结合性
1	() [] . 方法调用	左
2	后置 ++ --	左
3	一元 + - ! ~ 前置 ++ -- (T)强转	右
4	* / %	左
5	+ -(含字符串拼接)	左
6	<< >> >>>	左
7	< <= > >= instanceof	左
8	== !=	左
9	&	左
10	^	左
11		左
12	&&	左
13		左
14	?:	右
15	= += -= *= /= %= <<= >>= >>>= &= ^= =	右

与 C 的差异:新增 >>> (在 6 级)、instanceof(在 7 级)、+ 可用于字符串拼接。结论:把 C 的优先级表平移过来,99% 情况正确。工程惯例:复杂表达式加括号,优先级表只用于读代码。

2.6.3 求值顺序:从左到右,写进规范

示例 2-9 求值顺序演示

```
public class OrderDemo {
    public static void main(String[] args) {
        int r = 2 + 3 * 4;           // 14:优先级决定"先乘后加"
        int r2 = (2 + 3) * 4;        // 20:括号改变结合
        boolean b = 1 < 2 && 3 < 4; // true:比较优先于逻辑
        System.out.println(r + " " + r2 + " " + b);

        // 求值顺序:操作数从左到右求值(C 中是未定义行为!)
        int x = f() + g() * h();     // 输出 f g h:一定从左到右
        System.out.println(x);      // 1 + 2*3 = 7
    }
    static int f() { System.out.print("f "); return 1; }
    static int g() { System.out.print("g "); return 2; }
    static int h() { System.out.print("h "); return 3; }
}
```

⚠ 易错点 5:把 C 的 UB 习惯带进 Java

C 里 `a[i++] = i++` 是 UB;Java 里它是定义良好的(从左到右求值),但结果仍是反直觉的。规范规定求值顺序≠鼓励你写这种代码。生产铁律:一个表达式里最多一次自增/自减,副作用(赋值、调用)独立成句,这是 C 与 Java 通用的代码规范。

§2.7 字符串拼接与 Math 工具速览

2.7.1 "+" 的字符串魔法

Java 的 `+` 遇到字符串时执行拼接, 两侧任意类型都会被转成字符串; 拼接顺序从左到右, `1 + 2 + "3"` 是 `"33"`, `"1" + 2 + 3` 是 `"123"` ——这个差异是面试必考题。循环内大量拼接必须用 `StringBuilder`(第1章已述)。

2.7.2 Math 工具:比 C 的 `math.h` 更顺手

`Math.abs` (注意 `Integer.MIN_VALUE` 取绝对值仍是负数, 回绕!)、`Math.max/min` (C 要手写宏)、`Math.pow/sqrt/exp/log`、`Math.floor/ceil/round` (`round` 是"四舍五入"但 `Math.round(-1.5)` 得 `-1`, 向正无穷)、`Math.random()` (`0~1`)。整数运算专用 `Math.floorDiv/floorMod/addExact` 已在 §2.1 讲过。

示例 2-10 Math 速览与字符串拼接

```
public class MathDemo {
    public static void main(String[] args) {
        // 字符串拼接顺序陷阱
        System.out.println(1 + 2 + "3");    // 33:先算 1+2=3,再拼
        System.out.println("1" + 2 + 3);    // 123:从左到右都是拼

        // Math 常用
        System.out.println(Math.max(3, 5));    // 5
        System.out.println(Math.abs(-7));    // 7
        System.out.println(Math.abs(Integer.MIN_VALUE)); // -2147483648(回绕!)
        System.out.println(Math.pow(2, 10));    // 1024.0(double)
        System.out.println(Math.floor(-1.5));    // -2.0(向下)
        System.out.println(Math.round(-1.5));    // -1(向正无穷,注意)
        System.out.println((int) (Math.random() * 6) + 1); // 1~6 随机数
    }
}
```

工程建议:随机数用 `ThreadLocalRandom`

生产代码中 不要用 `Math.random()`: 它在多线程下有竞争且不支持指定范围。用 `ThreadLocalRandom.current().nextInt(1, 7)` (生成 `1~6`, 左闭右开)。需要密码学安全的随机用 `java.security.SecureRandom` ——这是安全规范的红线。

本章速查表

主题	要点 / 写法
int 溢出	补码回绕(不是 UB);long 参与运算避免;Math.addExact 检测溢出
整数除零	抛 ArithmeticException;浮点除零得 Infinity/NaN
/ 与 %	向零取整;符号跟随被除数(-7%2 = -1);floorDiv/floorMod 向下取整
自增自减	i++ 先用后变 / ++i 先变后用;一个表达式内最多用一次
复合赋值	i += x 自带强转:s += 1 合法,s = s + 1 编译错误;i += 0.9 截断为 5
位运算	& ^ ~ 与 C 相同;只能用于整数
移位	<< 左移;>> 算术右移(补符号位);>>> 逻辑右移(补 0);距离按 32/64 取模
逻辑运算	&& 短路;操作数必须是 boolean;! 取反;判空用 && 不用 &
比较	基本类型比值;引用比对象身份;字符串/包装类用 equals
浮点比较	0.1+0.2 != 0.3;用 a-b < eps 或转整数(分)比较;NaN 用 isNaN
instanceof	o instanceof String s:模式匹配,判断即声明;null 恒为 false
三元 ?:	只算选中的分支;两分支类型会自动找公共类型
求值顺序	操作数从左到右(C 是 UB);副作用独立成句
字符串 +	两侧任意类型转字符串;1+2+"3"="33","1"+2+3="123"
Math	max/min/pow/floor/ceil/round;abs(MIN_VALUE) 仍是负数;random 用 ThreadLocalRandom

最小必会清单

- 能解释"溢出回绕"与 C UB 的区别,并说出两种防护手段(long、Math.addExact)
- 能预测:整数除零、浮点除零、0.0/0.0 的三种行为
- 能说出 -7/2、-7%2、Math.floorDiv(-7,2)、Math.floorMod(-7,2) 四个结果
- 能解释 s += 1 与 s = s + 1 一个合法一个报错的原因
- 能写全三种移位并说出 >>> 的场景;知道 1 << 32 == 1
- 能用 && 写判空模式;知道 & 与 && 在 boolean 上的行为差异
- 能写出浮点比较的两种生产姿势(epsilon、整数化)
- 会写 instanceof 模式匹配并知道 null 的处理
- 能默写优先级表中 && 与 == 与 + 的顺序;能说出求值顺序的规定
- 能预测 1+2+"3" 与 "1"+2+3 的结果

自测题

Q 1. 写出输出: `int max = Integer.MAX_VALUE; System.out.println(max + 1); System.out.println(max + 1L);`

✓ 参考答案

第一行 -2147483648(int 回绕);第二行 2147483648(int 与 long 相加时 int 提升为 long,不溢出)。若想检测溢出,用 `Math.addExact(max, 1)` 它抛 `ArithmeticException`。

Q 2. 求值: `System.out.println(-7 / 2); System.out.println(-7 % 2); System.out.println(Math.floorMod(-7, 2));`

✓ 参考答案

-3(向零取整,不是向下);-1(符号跟随被除数,与 C 一致);1(floorMod 结果恒为非负)。生产上需要"非负余数"时用 floorMod,如环形索引 `Math.floorMod(i, size)`。

Q 3. 下面的代码编译通过吗?结果是什么? `short s = 1; s += 1; s = s + 1; int i = 5; i += 0.9;`

✓ 参考答案

`s += 1` 编译通过(自带强转), `s = s + 1` 编译失败(int 不能隐式收窄为 short); `i += 0.9` 编译通过且 i 变为 5(小数截断)。复合赋值 = 先运算 + 隐式强转,这是它"放过类型错误"的原因。

Q 4. 求值: `1 << 32`、`-16 >> 2`、`-16 >>> 2`

✓ 参考答案

1(移位距离按 32 取模,等价 `1 << 0`);-4(算术右移保留符号位);1073741820(逻辑右移高位补 0,即 `0x3FFFFFFC`)。记忆:>> 是"带符号除",>>> 是"无符号位搬运"。

Q 5. 判断对错: `double a = 0.1 + 0.2; a == 0.3` 是 true 还是 false?如何正确比较?

✓ 参考答案

false:0.1 与 0.2 的二进制近似相加得到 `0.30000000000000004`,与 0.3 字面量的二进制表示不同。正确做法: `Math.abs(a - 0.3) < 1e-9` (误差范围),或金额场景全部用 long 分为单位计算。

Q 6. 下面代码会除零崩溃吗? `int n = 0; boolean ok = (n != 0) && (10 / n > 1);` 如果把 && 换成 & 呢?

✓ 参考答案

用 && 不崩溃:左侧 `n!=0` 为 false,短路,右侧不执行。换成 & 会崩溃:& 不短路,两侧都执行, `10 / 0` 抛 `ArithmeticException`。判空 + 后续访问必须用 &&。

Q 7. 写出 `1 + 2 + "3"` 与 `"1" + 2 + 3` 的结果并解释。

✓ 参考答案

前者 "33":从左到右,先算 `1+2=3`(数值加),再与 "3" 拼接;后者 "123":第一个 + 遇到字符串就开始拼接,后续都是拼。记忆:拼接从"第一次遇到字符串"开始生效。

章末实验题:整数表达式计算器

实验 2:整数表达式计算器

实验目标:实现一个命令行表达式计算器,支持全章运算符并按 Java 优先级求值,正确处理短路。

实验要求:

- 任务 1:支持 $+$ $-$ $*$ $/$ $\%$ 四则与取模,遵循"乘除先于加减"(覆盖 §2.1 算术、§2.6 优先级)
- 任务 2:支持 $<<$ $>>$ 移位与 $\&$ $|$ $\wedge$ $\sim$ 位运算(覆盖 §2.3 位运算)
- 任务 3:支持 $==$ $!=$ $<$ $<=$ $>$ $>=$ 比较,结果为 0/1(覆盖 §2.5 比较)
- 任务 4:支持 $\&\&$ $||$ 逻辑运算,必须真正短路:右侧不再计算(覆盖 §2.4 短路)——注意 Java 的操作数必须是 boolean,这是"从 C 到 Java"的现场练习
- 任务 5:支持一元 $-$ $!$ $\sim$ 与括号(覆盖 §2.6 优先级、§2.2 单目结合性)
- 任务 6:输入 exit 退出;除零等错误用 try-catch 捕获并提示(覆盖 §2.1 除零语义)

实验环境:JDK 17+; `javac Calculator.java` 编译, `java Calculator` 运行。测试用例: $2+3*4=14$ 、 $(2+3)*4=20$ 、 $1<<4=16$ 、 $1+2==3 \ \&\& \ 5>3=1$ 、 $0 \ \&\& \ 1/0$ 不应报错。

第一步 实现"词法+语法"骨架: `expr/term/factor` 三层递归下降,只支持加减乘除与括号,跑通 $2+3*4$ 与 $(2+3)*4$

第二步 在正确的层级插入移位、位运算、比较与逻辑:优先级从低到高为 $|| \rightarrow \&\& \rightarrow | \rightarrow \wedge \rightarrow \& \rightarrow == \ != \rightarrow$ 关系 $\rightarrow$ 移位 $\rightarrow$ 加减 $\rightarrow$ 乘除 $\rightarrow$ 一元 $\rightarrow$ 括号,每一层一个方法

第三步 路实现:逻辑层用 `v != 0 && and() != 0 ? 1 : 0` 的写法——Java 的 `&&` 自带短路,左侧为假时右侧递归根本不会执行;用 `0 && 1/0` 验证

参考答案要点

```

import java.util.Scanner;

/**
 * 整数表达式计算器 — 第2章综合实验参考答案
 * 覆盖:算术/取模、位运算、移位、比较、逻辑短路、一元运算、优先级与括号
 * 真值规则:按 C 风格,非 0 即真,比较与逻辑的结果为 0 或 1
 */
public class Calculator {
    private final String src;
    private int pos;

    public Calculator(String src) { this.src = src; }

    private int expr() { return or(); } // 顶层

    // 优先级从低到高,每层一个方法(递归下降)
    private int or() {
        int v = and();
        while (match("||")) {
            v = (v != 0 || and() != 0) ? 1 : 0; // 短路:左侧真则右侧不执行
        }
        return v;
    }

    private int and() {
        int v = bitOr();
        while (match("&")) {
            v = (v != 0 && bitOr() != 0) ? 1 : 0; // 短路:左侧假则右侧不执行
        }
        return v;
    }

    private int bitOr() {
        int v = bitXor();
        while (match("|")) { v = v | bitXor(); }
        return v;
    }

    private int bitXor() {
        int v = bitAnd();
        while (match("^")) { v = v ^ bitAnd(); }
        return v;
    }

    private int bitAnd() {
        int v = equality();
        while (match("&")) { v = v & equality(); }
        return v;
    }

    private int equality() {
        int v = relational();
        while (true) {
            if (match("==")) { v = (v == relational()) ? 1 : 0; }
            else if (match("!=")) { v = (v != relational()) ? 1 : 0; }
            else { return v; }
        }
    }

    private int relational() {
        int v = shift();
        while (true) {
            if (match("<=")) { v = (v <= shift()) ? 1 : 0; }
            else if (match(">=")) { v = (v >= shift()) ? 1 : 0; }
            else if (match("<")) { v = (v < shift()) ? 1 : 0; }
            else if (match(">")) { v = (v > shift()) ? 1 : 0; }
            else { return v; }
        }
    }
}

```

```

}

private int shift() {
    int v = additive();
    while (true) {
        if (match("<<")) { v = v << additive(); }
        else if (match(">>")) { v = v >> additive(); }
        else { return v; }
    }
}

private int additive() {
    int v = multiplicative();
    while (true) {
        if (match("+")) { v = v + multiplicative(); }
        else if (match("-")) { v = v - multiplicative(); }
        else { return v; }
    }
}

private int multiplicative() {
    int v = unary();
    while (true) {
        if (match("*")) { v = v * unary(); }
        else if (match("/")) { v = v / unary(); } // 除零 -> ArithmeticException
        else if (match("%")) { v = v % unary(); }
        else { return v; }
    }
}

private int unary() {
    if (match("-")) { return -unary(); } // 一元负号(右结合)
    if (match("!")) { return (unary() == 0) ? 1 : 0; } // 逻辑非
    if (match("~")) { return ~unary(); } // 按位取反
    return primary();
}

private int primary() {
    if (match("(")) {
        int v = expr();
        match(")"); // 忽略不匹配的右括号
        return v;
    }
    return number();
}

private int number() {
    int v = 0;
    while (pos < src.length() && Character.isDigit(src.charAt(pos))) {
        v = v * 10 + (src.charAt(pos) - '0');
        pos++;
    }
    return v;
}

// 尝试匹配运算符;匹配成功则前进并返回 true
private boolean match(String op) {
    if (src.startsWith(op, pos)) {
        pos += op.length();
        return true;
    }
    return false;
}

public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    System.out.println("整数表达式计算器(输入 exit 退出)");
    System.out.println("支持: + - * / % << >> & | ^ ~ == != < <= > >= && || ! 括号");
    while (true) {

```

```

        System.out.print("> ");
        String line = in.nextLine().trim();
        if (line.equals("exit")) { break; }
        try {
            Calculator c = new Calculator(line);
            System.out.println("=" + c.expr());
        } catch (ArithmeticException e) {
            System.out.println("错误:整数除零抛 ArithmeticException(C 中这是 UB)");
        } catch (Exception e) {
            System.out.println("语法错误:" + e);
        }
    }
    in.close();
}
}

```

设计说明:递归下降是"优先级"最直接的代码化:每个优先级一层方法,方法体是"先解析上一层,再循环匹配本层运算符"。短路的关键在 `v != 0 && and() != 0`:Java 的 `&&` 保证左侧为假时右侧不执行,于是 `0 && 1/0` 不会除零——这同时演示了 §2.4 的"操作数必须是 boolean"这一 C 到 Java 的核心差异。除零由 `v / unary()` 抛出 `ArithmeticException`,在 `main` 中捕获提示。扩展方向:支持变量、支持 `double`、加入幂运算符与函数调用。

下一章预告:第3章 流程控制。表达式算清楚了,就要谈"程序怎么走"——Java 的 `if` 条件、`switch` 的新旧两代语法、三种循环与带标签的 `break/continue`。